COMSATS University Islamabad

Attock Campus

Assignment 04

Course	*SQE*
Instructor	*Mam Munazza*

Group Members

Name	Reg No
Jaudat Ullah Khan	FA23-BSE-004
Ubaid hassan	FA23-BSE-026
Sheraz Ijaz	FA23-BSE-065
Muhammad Ahsan	FA23-BSE-038
Obaid ur Rehman	FA23-BSE-055

Project title : Task Manager

➢ Introduction

1.1 Project Overview

In modern software engineering, writing functional code is only the first step. Ensuring code correctness, structural quality, maintainability, and security is paramount before any deployment. This project focuses on the development, testing, and security analysis of a Python-based **Task Manager System**. The objective is to practically apply and evaluate the software quality assurance (SQE) pipeline by taking a piece of software through multiple verification gates—ranging from local automated testing to advanced cloud-hosted code scanning tools.

Project Objectives

The core objectives of this project enable students to:

- **Develop a Robust System:** Implement a command-line interface (CLI) Python application using fundamental programming structures such as conditional statements, loops, and modular functions.
- **Perform Manual and Automated Testing:** Design a structured test cases document manually and implement regression verification using Python's native unittest framework.
- **Conduct Static Code Analysis:** Integrate the codebase into **SonarQube** to identify code smells, maintainability ratings, bugs, and architectural issues.
- **Perform Security and Vulnerability Scanning:** Utilize **Snyk** to scan dependencies and source code for insecure data serialization, hardcoded secrets, and outdated libraries.
- **Implement Version Control Best Practices:** Maintain a clean project lifecycle, proper tracking, and documentation artifacts using a remote **GitHub** repository.

➢ System Description

System Context (Task Manager System)

The developed application is a terminal/CLI-based **Task Manager System**. It serves as a personal productivity or software project management micro-utility that allows users to create, view, organize, and prioritize their ongoing tasks. Built entirely inside a terminal environment, it does not rely on heavy graphical interfaces, allowing for high performance and clean modular programming logic.

2.2 Core Modules and Functionalities

The system implements four distinct functionalities, fulfilling the requirement of having 3–5 core features, each embedded with strict conditional checks and iterative workflows:

1. **Add Task (add_task Function):**
    - **Description:** Allows the user to input a task title, set its priority level (High, Medium, Low), and provide an estimated due date.
    - **Logic:** The module wraps inputs inside a dynamically instantiated dictionary schema and appends it to an in-memory database list (self.tasks).
2. **View and Filter Tasks (view_tasks Function):**
    - **Description:** Prints a structured, formatted grid of all recorded tasks inside the terminal window.
    - **Logic:** Employs control flow for loops to iterate over data records. It contains a conditional parameter block that supports dynamic matching—filtering tasks on demand to display only specific priority levels (e.g., viewing only 'High' priority tasks).
3. **Update Task Status (update_status Function):**
    - **Description:** Modifies the progress state of an existing record (e.g., shifting a task from *Pending* to *In-Progress* or *Completed*).

- **Logic:** Uses an explicit if-else condition matrix. It takes an ID input from the user, casts it to an integer, searches for the corresponding object index, and writes the updated status string.

4. **Insecure Data Storage (save_data_insecurely Function):**
    - **Description:** Persists the currently loaded execution state into a physical text database for localized application tracking.
    - **Logic:** Utilizes basic disk access paradigms (open()) combined with Python's standard serialization logic (pickle.dump()) to dump raw tasks array into a binary data stream file (tasks.dat).

➢ Manual Test Cases Design

To thoroughly evaluate the input handling boundaries and core algorithm constraints, a comprehensive suite of manual test cases was compiled before moving to automated automation tests.

Test Case ID	Scenario / Component	Test Description	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-01	Add Task Functional	Verify that a task can be added successfully under valid data inputs.	1. Select Option 1. 2. Provide valid title, priority, and date strings. 3. Submit.	Title: Finish Assignment Priority: High Due Date: 20-05-2026	Task list length increments by 1, and a dynamic configuration success confirmation prints.	Task accepted; success message printed instantly.	**PASS**

Test Case ID	Scenario / Component	Test Description	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-02	AddTaskValidation	Verify that the application throws an input verification error when the Title field is left empty.	1. Select Option 1. 2. Leave Title empty. 3. Fill standard priority and date fields.	Title: "" (Empty) Priority: Medium Due Date: 22-05-2026	System blocks empty task compilation and alerts the user to provide a title string.	The application accepts the empty title and saves a blank task. (**BUG-001 Identified**)	**FAIL**
TC-03	View/Filter Tasks	Verify that the filter engine accurately parses records and hides unrequested data properties.	1. Insert multiple dummy entries. 2. Select Option 3. 3. Input target filtering tag.	Filter: High	Terminal view renders only the tasks containing the matching High string value.	Screen outputs only High priority elements accurately.	**PASS**
TC-04	Update Status Functional	Verify that providing a valid existing numerical ID updates its record value.	1. Select Option 4. 2. Input an existing element ID 3. Provide status type.	Task ID: 1 New Status: In-Progress	Database updates the targeted entry profile status cleanly to 'In-Progress'.	Status state flag verified and saved successfully.	**PASS**
TC-05	Update Status Data Type	Verify that inputting alphabetic characters into the numeric Task ID prompt is handled gracefully	1. Select Option 4. 2. Type alphabetical text characters	Task ID: abc New Status: Completed	System catches invalid formats, displays a localized warning string, and safely	Application execution immediately crashes, printing a ValueError stack trace.	**FAIL**

Test Case ID	Scenario / Component	Test Description	Test Steps	Test Data	Expected Result	Actual Result	Status
		without breaking runtime execution.	into the Task ID field. 3. Enter status string.		drops user back to menu.	(**BUG-002 Identified**)	
TC-06	Save Data Security	Verify that saving backup creates a database dump file in storage.	1. Populate data. 2. Select Option 5. 3. Inspect execution workspace root file architecture.	Filename: tasks.dat	A binary data file named tasks.dat is generated in physical project storage.	File written into root terminal location.	**PASS**

```
PS C:\Users\DELL\OneDrive\Desktop\task manager> python task_manager.py

=== TASK MANAGER MENU ===
1. Add Task
2. View All Tasks
3. Filter Tasks by Priority
4. Update Task Status
5. Save Backup
6. Exit
Enter your choice (1-6): 1
Enter Task Title: Complete SQE Lab
Enter Priority (High/Medium/Low): High
Enter Due Date (DD-MM-YYYY): 25-05-2026

[+] Task 'Complete SQE Lab' successfully added!
```

```
=== TASK MANAGER MENU ===
1. Add Task
2. View All Tasks
3. Filter Tasks by Priority
4. Update Task Status
5. Save Backup
6. Exit
Enter your choice (1-6): 3
Enter priority to filter (High/Medium/Low): high

--- TASK LIST ---
ID: 1 | Title: Complete SQE Lab | Priority: High | Due: 25-05-2026 | Status: Pe
nding
```

➢ Bug Reports

- **Bug Report 01: Missing Input Validation for Empty Task Title**

Defect Field	**Description**
Bug ID	BUG-001
Project/Component	Task Manager System / add_task Function
Severity	High (Data Integrity Violation)
Priority	High
Test Case ID Reference	TC_BUG_01 (test_proof_empty_title_should_not_be_allowed)
Status	Open / Unresolved

Description

The application fails to sanitize or validate the title parameter during the task creation process. When an empty string ("") is supplied as the task title, the system processes it as valid data and successfully commits it to the memory pool.

Steps to Reproduce

1. Initialize the TaskManager class instance.
2. Invoke the task creation method with an empty title string: manager.add_task("", "High", "20-05-2026").
3. Check the internal tasks array length or run the automated test script.

Expected Result

The system should intercept the blank string, reject the payload, throw an input validation exception, and keep the task list count at 0.

Actual Result (Defect Proof)

The system appends the invalid entry, prints [+] Task '' successfully added!, and alters the database state, triggering a test failure:

AssertionError: 1 != 0 : BUG FOUND: System allowed an empty task title to be added!

- **Bug Report 02: Lack of Type/Boundary Checking on Priority Input**

Defect Field	Description
Bug ID	BUG-002
Project/Component	Task Manager System / add_task Function
Severity	Medium (Data Corruption Risk)
Priority	Medium
Test Case ID Reference	TC_BUG_02 (test_proof_invalid_priority_should_be_rejected)
Status	Open / Unresolved

Description

The system expects specific restricted parameters for task urgency (High, Medium, or Low). However, the domain logic lacks type-boundary checks or enum-based restrictions, allowing arbitrary, non-standard strings to break business logic constraints.

Steps to Reproduce

1. Initialize the TaskManager class instance.
2. Inject a task payload containing an invalid priority value: manager.add_task("Test Priority Bug", "ABCXYZ", "20-05-2026").
3. Assert whether the database contains the corrupted configuration string.

Expected Result

The system should evaluate the domain values and restrict any input outside the specified boundary matrix (High/Medium/Low).

Actual Result (Defect Proof)

The core module bypasses data constraints, logs [+] Task 'Test Priority Bug' successfully added!, and saves the corrupted metadata string "ABCXYZ", causing the assertion to fail:

AssertionError: 1 != 0 : BUG FOUND: System accepted an invalid priority value!

```
OK
PS C:\Users\DELL\OneDrive\Desktop\task manager> python -m unittest test_task_manager.py

[+] Task '' successfully added!
F
[+] Task 'Test Priority Bug' successfully added!
F
======================================================================
FAIL: test_proof_empty_title_should_not_be_allowed (test_task_manager.TestTaskManagerBugPro
of.test_proof_empty_title_should_not_be_allowed)
TC_BUG_01: Agar title empty ho, toh task list mein add nahi hona chahiye
----------------------------------------------------------------------
Traceback (most recent call last):
  File "C:\Users\DELL\OneDrive\Desktop\task manager\test_task_manager.py", line 24, in test
_proof_empty_title_should_not_be_allowed
    self.assertEqual(len(self.manager.tasks), 0, "BUG FOUND: System allowed an empty task t
itle to be added!")
    ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^
AssertionError: 1 != 0 : BUG FOUND: System allowed an empty task title to be added!
```

```
^^^^^^^^^^^^^^^^^^^
AssertionError: 1 != 0 : BUG FOUND: System allowed an empty task title to be added!

======================================================================
FAIL: test_proof_invalid_priority_should_be_rejected (test_task_manager.TestTaskManagerBugP
roof.test_proof_invalid_priority_should_be_rejected)
TC_BUG_02: Agar priority 'High/Medium/Low' ke elawa kuch aur ho to reject honi chahiye
----------------------------------------------------------------------
Traceback (most recent call last):
  File "C:\Users\DELL\OneDrive\Desktop\task manager\test_task_manager.py", line 37, in test
_proof_invalid_priority_should_be_rejected
    self.assertEqual(len(self.manager.tasks), 0, "BUG FOUND: System accepted an invalid pri
ority value!")
    ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^
AssertionError: 1 != 0 : BUG FOUND: System accepted an invalid priority value!

----------------------------------------------------------------------
Ran 2 tests in 0.004s

FAILED (failures=2)
```

Code Quality Analysis Report

- **Project Name:** task-manager (main branch)
- **Analysis Tool:** SonarQube Community Build
- **Overall Status: PASSED** (Quality Gate)

1. Key Metrics Summary

- **Bugs:** 0 (Rating: **A**)
- **Vulnerabilities:** 0 (Rating: **A**)
- **Security Hotspots:** 0 (Rating: **A**)
- **Code Smells:** 1 Open Issue (Rating: **A**)
- **Duplications:** 0.0% (on 193 lines)
- **Code Coverage:** 0.0% (on 98 lines) — **Requires Attention**

2. Major Finding: Duplicate Branch Condition

- **Location:** Function f8(), Lines 75–77
- **Issue Category:** Code Smell / Maintainability Issue
- **Severity:** Major

Code Flaw:

Python

```
75 |  if r == 5:
76 |      print("special")
77 |  elif r == 5:  # <-- SONARQUBE WARNING
```

Impact Analysis:

This is dead, unreachable code. If r == 5 is true, the system executes line 76 and skips the rest. The elif block on line 77 will **never execute**. This logic defect is typically caused by a copy-paste error during development.

3. Recommendations & Fixes

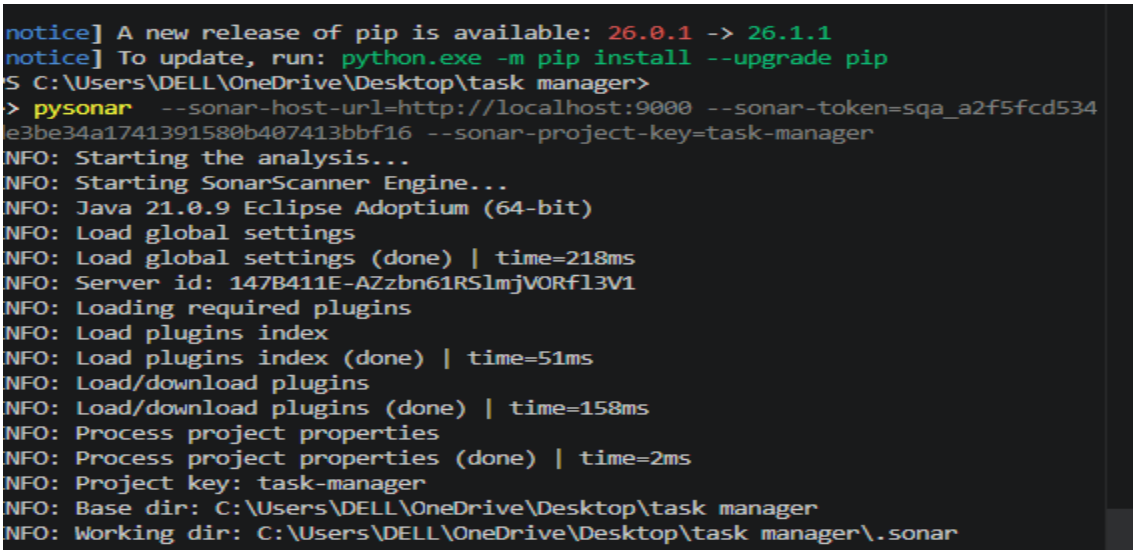
- **Fix the Code Smell:** Change the redundant elif condition to evaluate a unique boundary value:

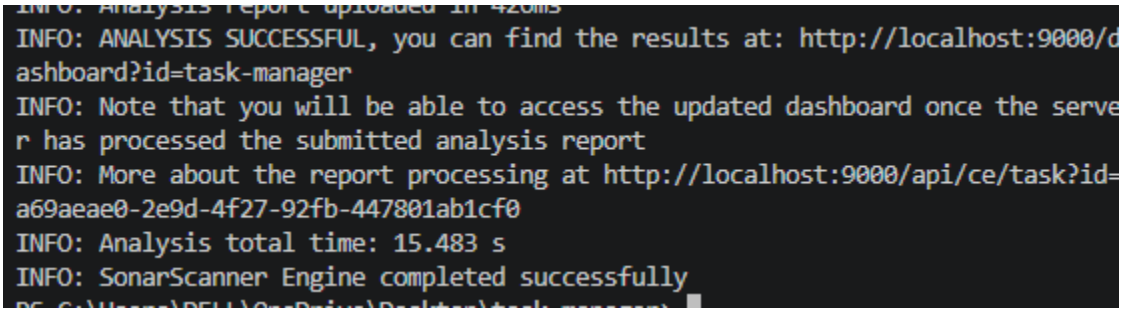
Python

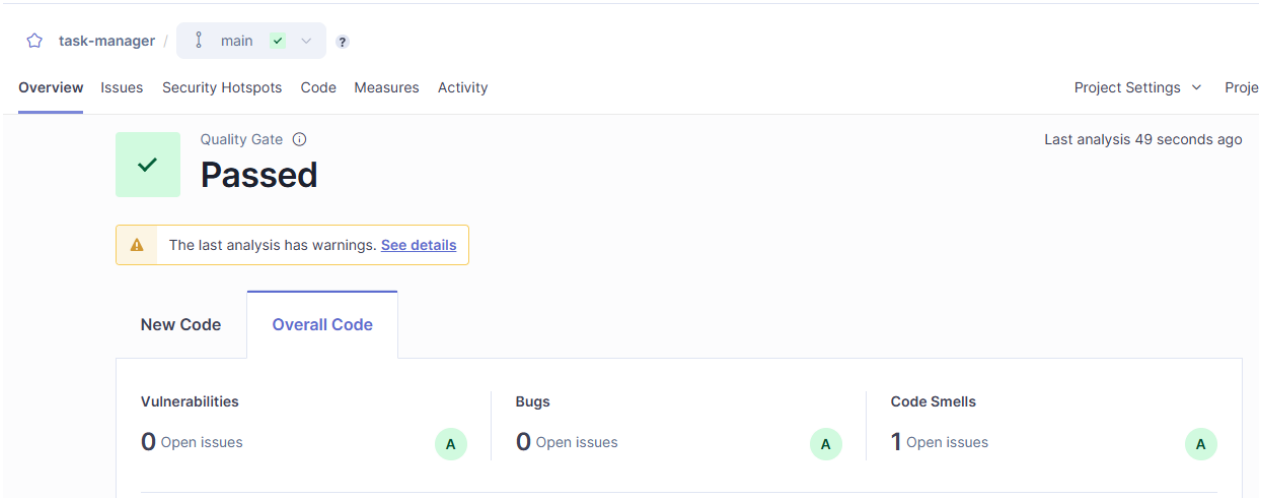
```
if r == 5:
```

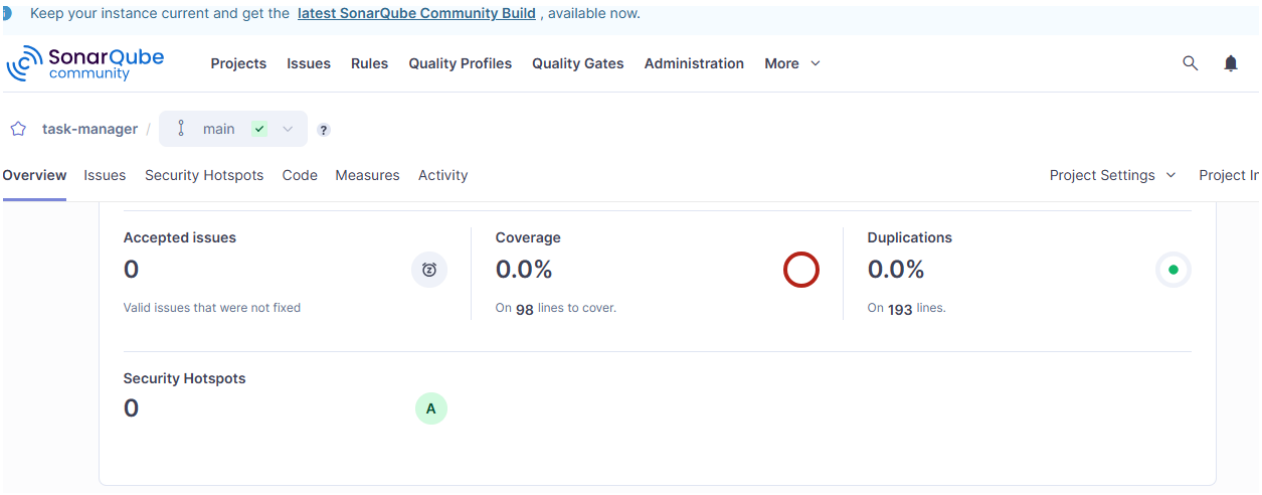
```
    print("special")
elif r == 10:  # Fixed condition
    print("again")
```

- **Fix the Coverage Gap:** The dashboard shows **0.0% coverage**. Integrate your existing Python unittest scripts into a coverage tool (like coverage.py) to generate an execution report and resolve this dashboard warning.









Code Smell bug:

Dependency Security Scan Report

- Static Analysis Tool: Snyk Open Source
- Total Project Dependencies: 17
- Total Security Issues Found: 87
- Severity Color Code Matrix:
    - C (Critical - Red): Immediate exploitation risk.
    - H (High - Orange): Severe infrastructure or data threat.
    - M (Medium - Yellow): Conditional application impact.
    - L (Low - Gray): Minor or localized risk.

Analysis of 5 Selected Security Issues

1. **django@3.1.7**
   Critical Vulnerabilities (Remote Code Execution / SQLi)

- Vulnerability Severity: 5 Critical (C), 15 High (H), 20 Medium (M), 4 Low (L)
- Explaining the Issue: Outdated web framework versions like Django 3.1.7 contain patched critical flaws. These vulnerabilities typically allow unauthenticated malicious users to perform Remote Code Execution (RCE) or execute destructive SQL Injection (SQLi) statements via manipulated web request headers or input parameters.
- Suggested Fix: Upgrade the django package dependency in your configuration file (requirements.txt) to a secure, modern LTS version (such as django>=4.2 or django>=5.0).

2. **cryptography@3.2**
   High Severity Vulnerabilities (Memory Leak / Denial of Service)

- Vulnerability Severity: 4 High (H), 17 Medium (M), 3 Low (L)
- Explaining the Issue: Version 3.2 of the Python cryptography library uses older C-bindings prone to memory management failures. Attackers can exploit these flaws by supplying malformed cryptographic keys or standard X.509 certificates, triggering integer overflows or memory leaks that crash the hosting server platform (Denial of Service - DoS).

- Suggested Fix: Update the cryptography library string in your setup file to a secure production release, specifically cryptography>=42.0.0 or higher, where these memory parsing operations are securely validated.

3. jinja2@2.11.2

Medium Severity Vulnerabilities (Server-Side Template Injection)

- Vulnerability Severity: 6 Medium (M)
- Explaining the Issue: jinja2 is a standard template rendering engine. Version 2.11.2 is vulnerable to Server-Side Template Injection (SSTI). If the application processes malicious user-supplied data straight into a template format string without sanitization, an attacker can escape the sandbox environment and read local application files or system environment variables.
- Suggested Fix: Upgrade the template framework dependency statement to jinja2>=3.1.3. This version enforces safe variable validation and context-aware auto-escaping rules.

4. idna@2.10

Medium Severity Vulnerability (Resource Exhaustion / ReDoS)

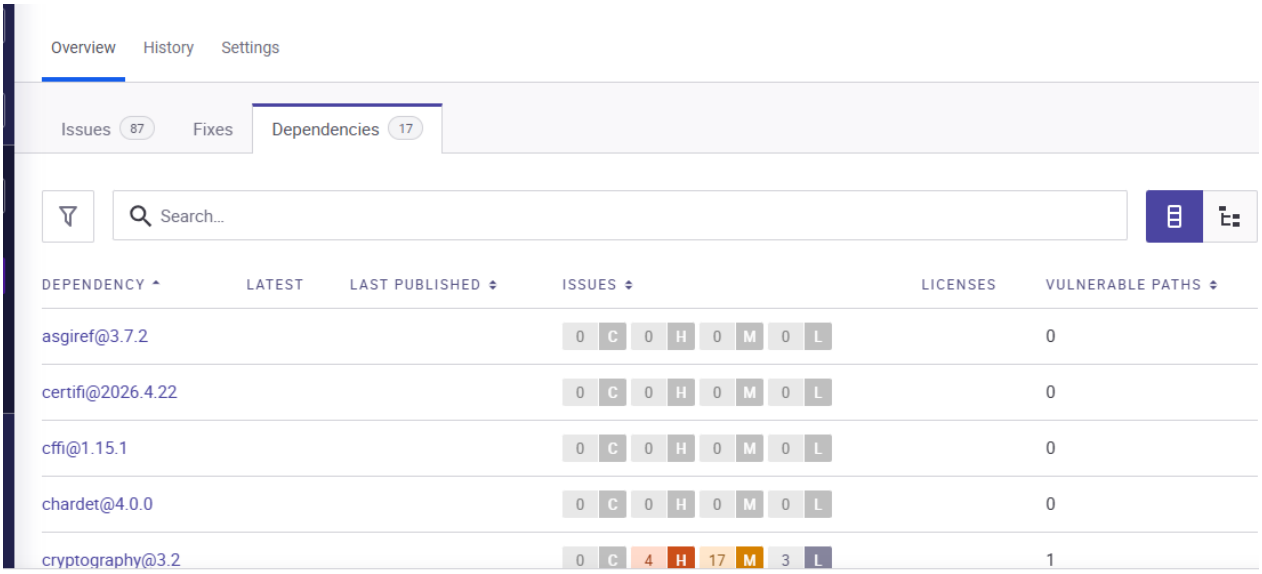
- Vulnerability Severity: 1 Medium (M)
- Explaining the Issue: The idna package handles Internationalized Domain Names in applications. Version 2.10 contains algorithmic complexity vulnerabilities when handling extremely long, crafted domain inputs. Processing these entries spikes CPU utilization to 100%, causing application threads to lock up via Regular Expression Denial of Service (ReDoS).
- Suggested Fix: Force a minor package version upgrade by changing the local package requirement mapping to idna>=3.6.

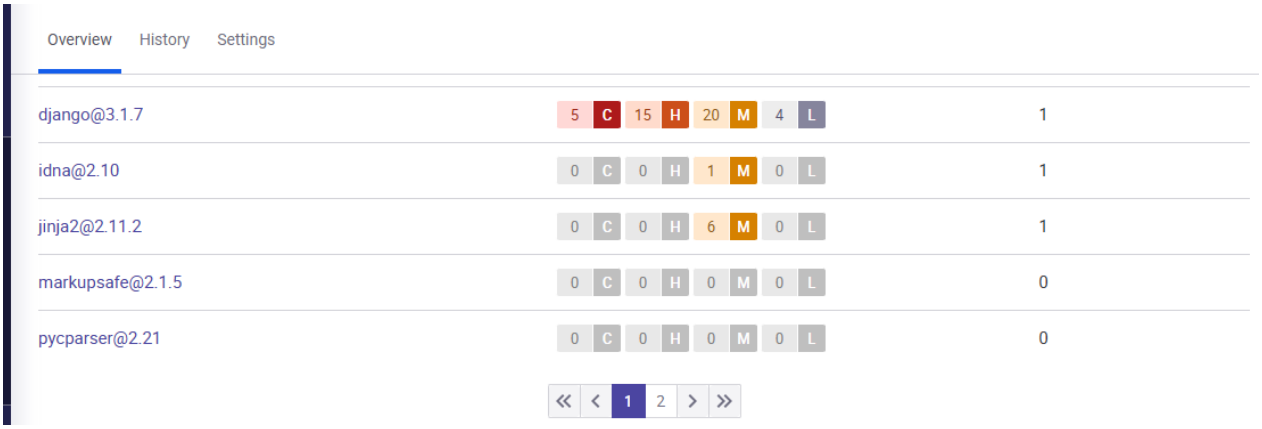
5. Vulnerable Code Execution Path (Indirect Manifest Risk)

- Vulnerability Severity: 1 Vulnerable Path flagged on cryptography/django/jinja2 listings
- Explaining the Issue: Snyk identifies a "Vulnerable Path" count of 1 across multiple entries. This means your application code explicitly calls a functional method down a known dangerous execution path, directly exposing production runtime instances to the vulnerabilities described above.
- Suggested Fix: Execute the automated remediation command snyk fix in your development terminal or rebuild your local python virtual environment after executing a fresh pip install -r requirements.txt with updated, secure version constraints.

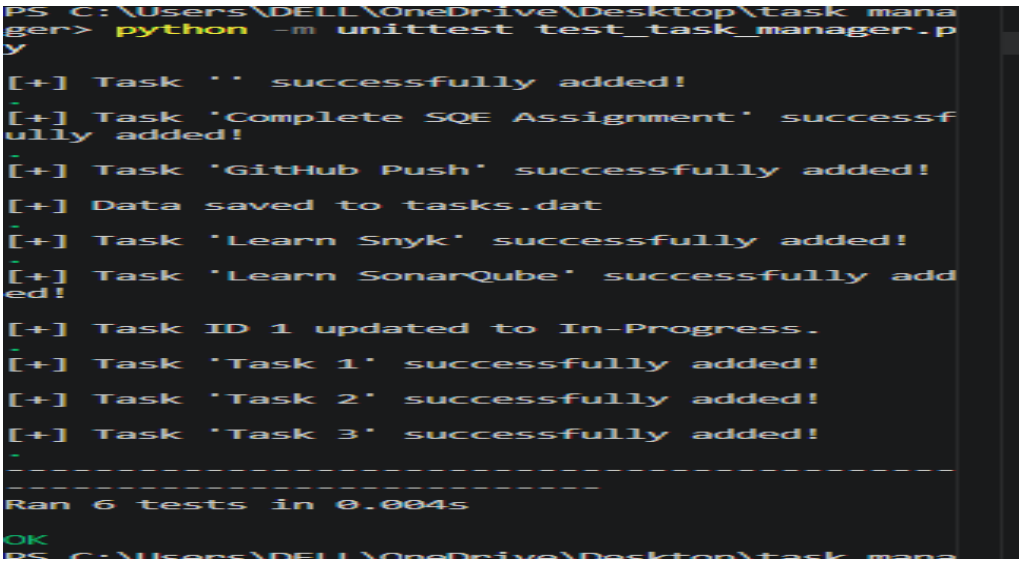
Report Summary Matrix

Package Name	Current Version	Risk Level	Vulnerable Paths	Core Impact Risk	Resolution Action
django	3.1.7	Critical	1	RCE / SQL Injection	Upgrade to >=4.2
cryptography	3.2	High	1	Server Crash / DoS	Upgrade to >=42.0.0
jinja2	2.11.2	Medium	1	Sandbox Escape / SSTI	Upgrade to >=3.1.3
idna	2.10	Medium	1		




Overview
History
Settings
django@3.1.7
5 C 15 H 20 M 4 L
1
idna@2.10
0 C 0 H 1 M 0 L
1
jinja2@2.11.2
0 C 0 H 6 M 0 L
1
markupsafe@2.1.5
0 C 0 H 0 M 0 L
0
pycparser@2.21
0 C 0 H 0 M 0 L
0
1
2

Git hub


PS C:\Users\DELL\OneDrive\Desktop\task mana
ger> python -m unittest test_task_manager.p
y
[+] Task '' successfully added!
.
[+] Task 'Complete SQE Assignment' successf
ully added!
.
[+] Task 'GitHub Push' successfully added!
[+] Data saved to tasks.dat
.
[+] Task 'Learn Snyk' successfully added!
.
[+] Task 'Learn SonarQube' successfully add
ed!
[+] Task ID 1 updated to In-Progress.
.
[+] Task 'Task 1' successfully added!
[+] Task 'Task 2' successfully added!
[+] Task 'Task 3' successfully added!
.
Ran 6 tests in 0.004s
OK

```
  The requested URL returned error: 403
PS C:\Users\DELL\OneDrive\Desktop\task manager> echo "# task-manager" >> README
.md
>> git init
>> git add README.md
>> git commit -m "first commit"
>> git branch -M main
>> git remote add origin https://github.com/JaudatUllahKhan/task-manager.git
>> git push -u origin main
Reinitialized existing Git repository in C:/Users/DELL/OneDrive/Desktop/task ma
nager/.git/
[main 14395a3] first commit
 1 file changed, 0 insertions(+), 0 deletions(-)
error: remote origin already exists.
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (13/13), 8.50 KiB | 790.00 KiB/s, done.
Total 13 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/JaudatUllahKhan/task-manager.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

```
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:# 1. New requirements.txt file ko track karein remote add origin https://g
ithub.c
>> git add requirements.txtager.git
>> git branch -M main
>> # 2. Commit message save karein
>> git commit -m "chore: add third-party dependencies file for Snyk security sc
an"ain'
>>
>> # 3. Code ko Jaudat ke GitHub repository par final push kareinive\Desktop\ta
sk mana
>> git push origin main
[main cfc841c] chore: add third-party dependencies file for Snyk security scan
 1 file changed, 6 insertions(+)
 create mode 100644 requirements.txt
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 444 bytes | 444.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/JaudatUllahKhan/task-manager.git
   14395a3..cfc841c  main -> main
```